A Framework for Probabilistic Oracles in Distributed Systems

Luca Lombardo

Abstract

Disaggregated systems require compact summaries of remote state to resolve the impedance mismatch between stateless compute and network-separated storage. We establish that the value of probabilistic data structures derives from two properties: dynamic mutability enables state synchronization through element deletion, while algebraic composability enables distributed aggregation through commutative operations. These properties address orthogonal constraints and impose conflicting requirements on state representation. We demonstrate through Cuckoo Filters and HyperLogLog that specific deployment patterns emerge as consequences of these properties, and derive decision criteria for structure selection based on workload characteristics.

1 Introduction

Disaggregated systems separate compute, memory, and storage into independent network-attached services. Compute workers are instantiated for short durations, executing stateless functions without persistent local state. Storage and memory servers maintain persistent state accessed over network interconnects. This separation creates an impedance mismatch: state-dependent computation must query remote state over network links with latencies orders of magnitude higher than local memory access. Every validation query that requires consulting remote state introduces a network round-trip cost that dominates application latency.

The mismatch manifests in two distinct operations. The first is membership testing: determining whether an element exists in a remote set before executing read-modify-write operations. A compute worker querying a deterministic key-value store incurs a network round-trip for every validation. The second is cardinality estimation: computing the number of distinct elements in a partitioned dataset [1]. An exact `COUNT(DISTINCT)` query requires a shuffle operation that transmits all unique elements to a central reducer [2]. Communication cost scales with dataset cardinality, rendering interactive analytics over petabyte-scale data infeasible.

Resolving this mismatch requires compact summaries that satisfy three conditions. The summaries must be queryable locally or transmitted with fixed cost independent of dataset size. They must synchronize with datasets that undergo insertion and deletion. They must support combination operations that produce results identical to processing the complete dataset when aggregating information across partitions.

Probabilistic data structures provide queryable summaries with sublinear space complexity. A Bloom filter occupies space proportional to the logarithm of the target false positive rate, independent of element size [3]. A HyperLogLog sketch estimates cardinality using fixed space regardless of dataset size [4]. These structures trade bounded error probability for space efficiency.

The architectural value of probabilistic structures in disaggregated systems derives from two properties that address different subsets of the three conditions above. Dynamic mutability addresses state synchronization: structures exhibiting this property support element deletion in constant time without auxiliary data structures, maintaining oracle consistency with datasets that undergo explicit removal operations. Composability addresses distributed aggregation: structures exhibiting this property support combination operations that are commutative, associative, and idempotent, enabling distributed reduction without central coordination or data shuffling [5].

In §4 we will see that these properties address orthogonal deployment constraints. Mutability ensures oracle state remains synchronized with evolving datasets by permitting efficient element removal. Composability enables distributed computation over partitioned data by supporting merge operations whose result is independent of merge order. The properties impose con-

flicting requirements on internal state representation. Mutability requires retaining sufficient information to identify and remove a specific element's contribution without affecting other elements. Composability requires information loss through commutative operations that discard element identities and enable order-independent aggregation.

This information-theoretic tension explains why specific structures optimize for different distributed contexts. Cuckoo Filters satisfy dynamic mutability through explicit fingerprint storage in deterministically computable locations, enabling targeted deletion without introducing false negatives for remaining elements. This property supports co-partitioned architectures where oracle state aligns with data partitioning, permitting local query execution. HyperLogLog satisfies algebraic composability through register-wise maximum operations that preserve the invariant property regardless of aggregation order, enabling hierarchical reduction architectures where partial results from multiple partitions combine into global aggregates.

1.1 Organization

Section 2 analyzes membership testing in distributed systems. We derive the requirement for dynamic mutability from state synchronization constraints and examine the Cuckoo Filter as a structure satisfying this property. We document deployment patterns enabled by mutability, including co-partitioned architectures, capacity scaling through consistent hashing, and concurrency control mechanisms.

Section 3 analyzes cardinality estimation in data-parallel systems. We derive the requirement for composability from distributed aggregation constraints and examine HyperLogLog as a structure satisfying this property. We document deployment patterns enabled by composability, including hierarchical reduction in MapReduce and stream processing systems, fault-tolerance protocols based on approximate recovery, and challenges arising from adversarial manipulation and privacy leakage in federated environments.

Section 4 formalizes the mutability-composability framework. We prove that idempotent merge operations and targeted deletion without false negatives are incompatible when reversible element representations are absent. We classify workloads by data lifecycle dynamics and query result formation, deriving decision criteria that map each workload class to the required property and corresponding architectural pattern.

Section 5 synthesizes the framework implications for disaggregated system design. We identify research directions at the intersection of the core properties with Byzantine resilience, differential privacy, and adaptive fault tolerance.

2 Set Membership

2.1 Probabilistic Membership Testing in Distributed Systems

We consider the dictionary problem, which consists of maintaining a dynamic set $S \subseteq \mathscr{U}$ and supporting membership queries. This capability is a fundamental primitive for distributed protocols concerned with set reconciliation: the process of computing the symmetric difference between sets held on disparate nodes. In a large-scale distributed environment, classical deterministic solutions to the membership problem, such as hash tables, impose space requirements proportional to both the number of elements and their individual sizes. For n elements of size $|x|$ bits, a hash table requires $\Theta(n \cdot |x|)$ space. When data is partitioned across nodes, the communication cost of synchronizing these structures scales with the same factor. For large objects or high-cardinality sets, this cost renders deterministic approaches infeasible in bandwidth-constrained systems [6, 7].

Probabilistic data structures resolve this constraint by trading a quantifiable measure of precision for substantial gains in space and communication efficiency. A Bloom filter, the canonical instance of this class, requires $\Theta(n \log(1/\varepsilon))$ space independent of element size, where ε denotes the target false positive rate. This sub-linear space complexity enables the design of systems that prioritize performance and availability over strict consistency.

2.1.1 Membership Queries in Large-Scale Systems

A one-sided error membership oracle guarantees correctness for negative answers. A definitive negative answer allows a system to avoid expensive, high-latency operations, such as disk I/O or network round-trips, which are often the primary bottlenecks to performance [8, 7]. For instance, a storage system may con-

sult such an oracle to determine if a specific data block might exist on a remote disk before initiating a costly read operation; a negative response obviates the need for any disk access [7, 8].

Definition 2.1 (One-Sided Error Membership Oracle)**.** A data structure $\mathscr{D}$ representing a set S is a one-sided error membership oracle with a false positive rate ε if for any element $x \in S$, the probability that $\mathscr{D}.\text{QUERY}(x)$ returns true is 1, and for any element $y \notin S$, the probability that $\mathscr{D}.\text{QUERY}(y)$ returns true is at most ε.

By accepting a bounded and well-defined error probability, these algorithms enable a level of scalability and performance that would be unattainable with exact, deterministic methods [8, 7].

2.1.2 The Standard Bloom Filter

The Bloom filter is the canonical implementation of a one-sided error membership oracle [3]. It achieves its space efficiency by storing compact representations of elements, derived from hash functions, rather than the elements themselves.

Definition 2.2 (Bloom Filter [3])**.** A Bloom filter for a set S with expected cardinality $|S| \leq n$ from a universe $\mathscr{U}$ consists of a bit array B of length m initialized to all zeros, and a collection of k independent hash functions $\mathscr{H} = \{h_1, \ldots, h_k\}$, where each $h_i : \mathscr{U} \to \{0, 1, \ldots, m-1\}$.

To insert an element $x \in S$, we set the bits at positions $B[h_i(x)]$ to 1 for all $i \in \{1, \ldots, k\}$. To query for an element y, the operation returns true if and only if $B[h_i(y)] = 1$ for all $i \in \{1, \ldots, k\}$. A query for any element $y \in S$ is guaranteed to return true, as all corresponding bits will have been set during its insertion. This structure therefore satisfies the no-false-negatives condition of the oracle by construction.

A false positive occurs for an element $y \notin S$ if all k bits corresponding to its hash values have been incidentally set to 1 by the insertion of other elements. Assuming the hash functions distribute elements uniformly over the bit array, the probability of a specific bit remaining 0 after the insertion of n elements is

$$(1 - 1/m)^{kn} \approx e^{-kn/m}$$

The probability of a false positive, ε, represent the probability that all k bits for the queried element are 1.

Proposition 2.3 (False Positive Probability [8])**.** *The false positive probability of a Bloom filter is*

$$\varepsilon \approx (1 - e^{-kn/m})^k$$

A given ratio m/n of bits per element determines an optimal number of hash functions that minimizes this probability.

Theorem 2.4 (Optimal Hash Function Count [9])**.** *For a given ratio m/n, the false positive rate ε is minimized when the number of hash functions is $k = (m/n)\ln 2$.*

This theorem yields a direct relationship between the space allocated per element and the achievable false positive rate.

Corollary 2.5 (Space-Error Relationship [9])**.** *To achieve a target false positive rate ε with an optimal number of hash functions, the required number of bits per element is*

$$m/n = -\frac{\ln \varepsilon}{(\ln 2)^2} \approx 1.44 \log_2(1/\varepsilon)$$

The asymptotic space complexity of the Bloom filter is therefore $\Theta(n \log(1/\varepsilon))$, independent of the size of the elements. The performance relies on high-quality hash functions. In practice, computationally inexpensive non-cryptographic hash functions are sufficient, and techniques such as double hashing, which generates k hash values from two initial hashes, can reduce computational overhead without asymptotically increasing the false positive probability [8].

The properties established above hold under two assumptions: the set S is static after construction, and its cardinality n is known when the filter is initialized. Distributed deployment introduces constraints that violate these assumptions. In systems where compute and storage are separated by network boundaries, where data undergoes continuous insertion and deletion across partitions, and where dataset cardinality is neither fixed nor predictable, the architectural requirements for membership oracles extend beyond the space-error trade-off. The subsequent analysis identifies these constraints and examines their implications for oracle design.

2.2 The Constraint of State Synchronization

In a distributed system where compute and storage are separated by network boundaries, a membership oracle must maintain coherence with a dataset

that evolves through insertion and deletion operations across multiple partitions. This requirement for state synchronization introduces constraints not present in single-node deployments where the oracle and the data it represents are co-located. We identify two failure modes that emerge when a membership oracle cannot adapt its internal state to reflect changes in the underlying dataset.

The first failure mode is the persistence of stale positives. When an element is deleted from the dataset, a Bloom filter that represents that dataset cannot remove the element's contribution from its bit array. The bits set during the element's insertion remain set indefinitely. Subsequent queries for the deleted element will continue to return true. In a distributed key-value store that uses Bloom filters to avoid disk I/O for nonexistent keys, this behavior forces the system to perform expensive read operations merely to discover that the key has been deleted or that a tombstone marker is present. The filter fails to converge to the true state of the system. The cost of this divergence is proportional to the delete rate of the workload; systems with high churn experience continuous wasted I/O as filters accumulate obsolete positive indications [10, 8].

The second failure mode is capacity exhaustion. A Bloom filter is parameterized for an expected cardinality n. When the actual number of inserted elements significantly exceeds this estimate, the bit array becomes saturated. As saturation increases, the false positive rate degrades toward unity, rendering the filter ineffective as a negative filter. In distributed environments where dataset cardinality is determined by aggregate insertion rates across multiple nodes and where traffic patterns are non-stationary, the capacity of a fixed-size filter becomes a constraint that requires either over-provisioning (wasting memory) or periodic reconstruction (introducing service interruption) [11, 8].

2.2.1 Variants and Trade-offs

We analyze two extensions to the standard Bloom filter that address the failure modes identified in 2.2. Each extension resolves one constraint but introduces a new limitation that restricts its applicability in distributed contexts.

The Counting Bloom Filter [12] addresses deletion by replacing the bit array with an array of counters. Each counter tracks the number of elements mapped to its position. Insertion increments the relevant counters; deletion decrements them. A query returns true if all corresponding counters are positive. This mechanism permits element removal without introducing false negatives for other elements.

Definition 2.6 (Counting Bloom Filter [12])**.** A Counting Bloom Filter consists of an array C of m counters, each of c bits, initialized to all zeros. It uses a collection of k independent hash functions $\mathcal{H} = \{h_1, \ldots, h_k\}$, where each $h_i : \mathcal{U} \to \{0, \ldots, m-1\}$.

The structure supports deletion but incurs a space overhead proportional to the counter width c. Analysis [12, 8] demonstrates that 4-bit counters suffice to prevent overflow for most workloads, yielding an approximately four-fold increase in memory consumption relative to a standard Bloom filter. The arithmetic properties of counter arrays enable set difference operations through counter subtraction, a capability useful for distributed set reconciliation protocols [13].

The Scalable Bloom Filter presented in [11], on the other hand, addresses the problem of capacity exhaustion through a sequence of constituent filters with increasing sizes and decreasing error rates. When the active filter reaches a load threshold, a new, larger filter is appended to the sequence. Queries then must inspect all filters in the sequence.

Definition 2.7 (Scalable Bloom Filter [11])**.** A Scalable Bloom Filter (SBF) is an ordered sequence of s Bloom filters, $B_1, B_2, \ldots, B_s$. Each filter B_i is defined by its size m_i and number of hash functions k_i. The sequence is constructed such that for $i > 1$, $m_i > m_{i-1}$ and the target false positive rate $\varepsilon_i < \varepsilon_{i-1}$.

This architecture accommodates unbounded growth but sacrifices a property critical for distributed aggregation. The heterogeneity of the constituent filters prevents bitwise operations such as the OR operation required for computing the union of two sets. Two SBF instances representing different sets cannot be merged through element-wise combination of their internal arrays because the arrays differ in size and hash function count. This absence of composability renders the SBF unsuitable for distributed query processing where partial results computed on different nodes must be aggregated [6, 11].

2.2.2 Dynamic Mutability

The analysis in 2.2 exposes a fundamental requirement for membership oracles in distributed systems. An oracle that cannot delete elements accumulates stale state, forcing redundant operations that negate the performance benefits the oracle was designed to provide. An oracle that cannot scale capacity requires either over-provisioning or service interruption for reconstruction. The CBF (2.6) resolves deletion at the cost of substantial space overhead. The SBF (2.7) resolves capacity at the cost of lost composability.

A membership oracle suitable for distributed deployment must satisfy three conditions: it must support element deletion in constant time, it must not require auxiliary data structures whose space overhead negates the oracle's compactness, and it must not introduce false negatives for elements that remain in the set. We define this requirement as the property of dynamic mutability.

Definition 2.8 (Dynamic Mutability)**.** A probabilistic oracle $\mathscr{D}$ representing a set S exhibits dynamic mutability if it supports both $\textsc{Insert}(x)$ and $\textsc{Delete}(x)$ operations for an element x in $O(1)$ time. The space complexity must increase by at most a constant multiplicative factor compared to a static probabilistic oracle with the same target false positive rate and expected capacity. The $\textsc{Delete}(x)$ operation must not introduce false negatives for any other element $y \in S, y \neq x$.

This property represent a requirement for oracles that must track the lifecycle of data in systems where deletion is a first-class operation. Distributed key-value stores that implement tombstones for deleted keys, Log-Structured Merge-Trees that compact data files and purge obsolete entries, and content distribution networks that expire cached objects all require membership oracles whose internal state can synchronize with these deletion events. The property of dynamic mutability characterizes the class of oracles that satisfy this requirement.

2.3 Cuckoo Filters

The Cuckoo Filter, introduced in [14], satisfies the conditions of dynamic mutability (Definition 2.8). Its delete operation executes in $O(1)$ time, requires no auxiliary counters, and does not introduce false negatives for other elements. This property derives from its partial-key hashing scheme (§2.3.1), which enables the relocation and removal of an element's fingerprint using only the fingerprint itself, decoupling the operation from access to the original element.

2.3.1 Partial-Key Hashing

The main challenge in deploying Cuckoo Filters across network boundaries lies in the relocation operation itself. During insertion, when both candidate buckets are full, the filter must displace an existing fingerprint to its alternate location. Standard cuckoo hashing accomplishes this by recomputing the hash of the original element; the element's full key must be available to determine where to move it. In a distributed system, however, filters may be partitioned or replicated across nodes, and the original element may not be co-located with the filter metadata. Retrieving the full element for relocation decisions across network boundaries would introduce prohibitive overhead.

The solution is partial-key cuckoo hashing, which enables relocation decisions using only the stored fingerprint. This capability is the prerequisite for all subsequent distributed patterns.

Definition 2.9 (Partial-Key Cuckoo Hashing [14])**.** For an element $x \in \mathscr{U}$ with precomputed fingerprint $f = \textsc{fingerprint}(x)$, the two candidate bucket indices are computed as:

$$i_1 = \textsc{hash}(x), \quad i_2 = i_1 \oplus \textsc{hash}(f)$$

where $\oplus$ denotes bitwise XOR. This allows us to, given i_2 and f, recover i_1 via the same equation:

$$i_1 = i_2 \oplus \textsc{hash}(f)$$

This symmetry decouples relocation from full-key access. When a fingerprint must be moved from bucket i_1 to its alternate location, the system computes the destination using only f and the current position. The fingerprint can traverse network boundaries, be temporarily replicated on different nodes, or be moved as part of partition rebalancing, all without requiring access to the original element. For distributed systems where filters may be stored separately from data, or where data undergoes continuous replication and migration, this property is essential.

Definition 2.10 (Cuckoo Filter Operations [14])**.** A Cuckoo Filter with m buckets, each containing b entries, supports three operations:

Lookup(x): Compute $i_1 = \text{HASH}(x)$, $i_2 = i_1 \oplus \text{HASH}(f)$. Return true if fingerprint f exists in either bucket i_1 or i_2; otherwise return false. Query always examines at most $2b$ entries.

Insert(x): Compute i_1, i_2. If either bucket contains an empty entry, place f there and complete. Otherwise, select one bucket randomly, displace its resident fingerprint f', compute the alternate location for f' using $i \leftarrow i \oplus \text{HASH}(f')$, and repeat the displacement process. If relocations exceed a configurable threshold (typically 500), insertion fails and a capacity increase is required.

Delete(x): Compute i_1, i_2. If f exists in either bucket, remove one copy; otherwise return failure. Deletion requires no counters or additional bookkeeping.

A false positive here occurs when an element not in the set hashes to a fingerprint that collides with an existing fingerprint in one of its two candidate buckets. A query inspects at most $2b$ entries across two buckets. This inspection process yields a direct upper bound on the false positive probability.

Proposition 2.11 (False Positive Bound [14])**.** *Let a Cuckoo Filter be configured with b entries per bucket and a fingerprint length of f bits. The false positive probability ε satisfies the inequality:*

$$\varepsilon \leq \frac{2b}{2^f}$$

From this inequality, we can derive the minimum fingerprint size f required to achieve a target false positive probability ε:

$$f \geq \lceil \log_2(1/\varepsilon) + \log_2(2b) \rceil$$

For a configuration with $b = 4$ and a target $\varepsilon = 0.01$ (a 1% false positive rate), this relationship requires a fingerprint size of at least $f \geq \lceil \log_2(100) + \log_2(8) \rceil = \lceil 6.64 + 3 \rceil = 10$ bits.

The architectural consequences of this design become evident when we consider the delete operation. A standard Bloom filter offers no delete mechanism; removing an element's bits risks corrupting the representation of other stored elements. A Counting Bloom Filter enables deletion but requires per-position counters, which results in a space overhead of approximately four times that of a standard Bloom filter [14]. In contrast, the Cuckoo Filter implements deletion by locating the specified fingerprint in one of its two candidate buckets and removing it. This operation requires no auxiliary data structures and directly enables the dynamic distributed architectures.

2.3.2 Capacity Scaling

In a distributed system, the cardinality of a dataset is not known *a priori*, requiring any data structure to accommodate significant growth. A Cuckoo Filter of a fixed size cannot accept new insertions once its load factor approaches its theoretical limit. While single-node deployments address this by rebuilding the filter into a larger table offline, such a procedure is untenable in distributed systems that operate under strict availability and latency constraints. A node that halts operations to rebuild its local filter violates service agreements and introduces a point of coordination that undermines distributed design principles.

The Dynamic Cuckoo Filter (DCF), introduced by Chen et al. [15], presents a systematic solution to this scaling problem. Instead of rebuilding a full filter, the DCF architecture allocates a new, empty filter and appends it to an ordered sequence of filters. All new insertions are directed to the most recently appended filter.

Definition 2.12 (Dynamic Cuckoo Filter [15])**.** A DCF is an ordered sequence of Cuckoo Filters, $\text{CF}_1, \text{CF}_2, \ldots, \text{CF}_k$. A pointer, curCF, references the terminal filter in the sequence. An insertion operation first attempts placement on curCF. If this insertion fails due to capacity limits, a new filter is allocated, appended to the sequence, and designated as the new curCF. A query operation must traverse this sequence in reverse chronological order, from curCF to CF_1, until the target fingerprint is found or all filters have been examined.

The DCF architecture permits unbounded capacity growth without service interruption, but it achieves this at the cost of query performance. A query for an element x requires a sequential search through the filter chain. If x was inserted into an early filter and the chain has subsequently grown to a length of k, the query must inspect all k filters.

Proposition 2.13 (DCF Query Complexity)**.** *A membership query in a DCF composed of k constituent filters requires $O(k)$ bucket accesses in the worst case. For a system containing n total items, where each filter maintains a load factor $\alpha \approx 0.95$ and has a capacity of C items, the number of filters scales as $k = \Theta(n/C)$. Consequently, query latency grows proportionally with the total dataset size.*

This pattern creates a direct conflict between capacity scaling and query performance. The mechanism that enables system growth is the same mechanism that causes query latency to degrade.

This linear query complexity arises because the DCF organizes its constituent filters temporally, by insertion order, rather than deterministically by element hash value. No direct mapping exists between an element's content and the specific filter that contains it. Membership testing therefore degenerates to a sequential search.

2.3.3 Consistent Hashing

The linear degradation in query performance of the Dynamic Cuckoo Filter arises from its temporal organization. We observe that distributed systems have already addressed analogous scaling challenges for data partitioning. Architectures such as Apache Cassandra [16] and Amazon DynamoDB [16] employ consistent hashing to distribute their keyspace. In these systems, nodes are assigned token ranges on a virtual hash ring. An element's hash value deterministically maps it to a specific point on the ring, and thus to a responsible node. The addition or removal of a node requires remapping only a localized subset of the keyspace, preserving a stable and deterministic mapping for the majority of data. Redis Cluster [17] achieves a similar outcome using a hash slot partitioning scheme, where a fixed number of slots are distributed among nodes and can be migrated individually to scale the cluster.

This same principle can be applied to the organization of a Cuckoo Filter's buckets. Instead of organizing filters in a temporal sequence, we can distribute the buckets themselves across a consistent hash ring.

Definition 2.14 (Index-Independent Cuckoo Filter [18])**.** An Index-Independent Cuckoo Filter (I2CF) maintains its buckets on a consistent hash ring. For an element x with fingerprint f, its candidate bucket locations are determined by hashing the element to points on this ring. The system identifies the first bucket successor to each of these points on the ring as a candidate bucket.

The core property of the I2CF is that bucket identifiers depend only on the element's hash value, not on the total number of buckets m in the system. Capacity expansion is achieved by adding a new bucket to a chosen position on the ring. This operation necessitates relocating only those elements whose hash values fall within the range immediately preceding the new bucket's position. For a system with n items distributed across m buckets, a rebalancing event affects an expected $O(n/m)$ items, a small fraction of the total dataset.

This design decouples capacity growth from query complexity, resolving the conflict present in the DCF (2.12)

Theorem 2.15 (I2CF Query Complexity)**.** *For an I2CF with m buckets organized on a consistent hash ring, a membership query for an element x requires a sequence of constant-time operations: (1) hash computations to determine ring positions, (2) a ring lookup to identify candidate buckets (achievable in $O(1)$ expected time with an appropriate auxiliary index), and (3) inspection of entries within those buckets. The total query complexity is therefore $O(1)$, independent of the total number of items n and the total number of buckets m.*

We saw that the DCF modifies the element-to-filter mapping to accommodate growth, leading to performance degradation. In contrast, the I2CF maintains a fixed, deterministic mapping from elements to ring positions while allowing the set of buckets to expand elastically. As a result, growth and query performance become orthogonal concerns.

2.3.4 Co-Partitioning Filters with Distributed Data

We now observe that the value of the I2CF (2.14) is most fully realized when its partitioning scheme is aligned with that of the host distributed system. Many distributed key-value stores partition their keyspace across a set of nodes using a consistent hash ring. In such systems, each node assumes responsibility for one or more token ranges on the ring.

We apply this same partitioning logic to the filter's buckets, an approach whose efficacy is demonstrated by Luo et al. in their work on distributed filters over Redis Cluster [19].

Definition 2.16 (Co-Partitioned Filter Architecture [19])**.** A distributed system of m nodes maintains a consistent hash ring, where each node n_i is assigned responsibility for a token range $[t_i, t_{i+1})$. The node maintains: (1) all data items d such that $H(d) \in [t_i, t_{i+1})$, and (2) all Cuckoo Filter buckets b such that position$(b) \in [t_i, t_{i+1})$. A membership query for an ele-

ment x is routed to the node responsible for the token range containing $H(x)$ for local processing.

The ability to execute deletion locally permits the stable assignment of filter buckets to consistent hash ranges, as element removal does not require global state coordination. A query processed on the node that owns the element's token range incurs only the cost of local memory access. A query originating from a different node requires a single network round-trip. The latency difference between local memory access and network communication is typically three to four orders of magnitude.

We can model the expected query latency for this architecture under a uniform query distribution. In a system with m nodes, a randomly routed query has a probability $1/m$ of being processed locally and a probability $(1-1/m)$ of requiring network transit. This leads to the following formulation for the expected query latency.

Theorem 2.17 (Co-Partitioned Query Latency)**.** *In a co-partitioned I2CF architecture of m nodes under a uniform query distribution, the expected query latency $\mathbb{E}[L_q]$ is given by:*

$$\mathbb{E}[L_q] = \frac{1}{m} t_{local} + \left(1 - \frac{1}{m}\right) t_{network}$$

where t_{local} represents local bucket access time and $t_{network}$ represents network round-trip latency.

This latency bound is independent of the total dataset size n. Query performance is determined by hardware characteristics and system scale (m), not by the number of elements stored.

By reusing an existing consistent hashing ring, the system gains a scalable membership testing capability with minimal additional coordination overhead. Fault tolerance mechanisms, such as data replication, can be extended to include filter buckets, allowing replicated filter state to propagate through existing replication channels.

The experimental results in [19] confirm the efficacy of this approach, demonstrating significant memory savings over Bloom filter alternatives, full support for deletions, and seamless capacity scaling as nodes are added to the cluster.

2.3.5 Hierarchical Elasticity

While the I2CF architecture provides fine-grained, bucket-level elasticity, its rebalancing mechanism is most effective under predictable, incremental growth in dataset cardinality. Distributed systems, however, often experience non-uniform growth patterns, including sudden spikes in insertion rates. In such scenarios, where the rate of cardinality growth exceeds the rate at which new buckets can be added and populated, the system may experience cascading insertion failures.

To address this class of workloads, we analyze the Consistent Cuckoo Filter (CCF), an architecture that introduces a second, coarser tier of elasticity by organizing a collection of independent I2CF instances [18].

Definition 2.18 (Consistent Cuckoo Filter [18])**.** A CCF is a collection of I2CF instances, $\{I_1, I_2, \ldots, I_s\}$, initially with $s = 1$. Insertions are directed to a designated active instance, typically I_s. When I_s reaches a predefined load factor threshold, a new, empty instance I_{s+1} is allocated and becomes the active target for subsequent insertions. A query operation must inspect all instances in the collection until the target fingerprint is found.

This architecture allows the system to increase its total capacity instantly by adding a new, untapped I2CF. However, this *scale-out* mechanism, if left unmanaged, structurally resembles the chaining model of the DCF (2.12). A query must probe every instance in the collection, leading to a performance dependency on the number of instances, s.

Proposition 2.19 (CCF Unmanaged Query Complexity)**.** *In a CCF where new, fixed-capacity I2CF instances are added sequentially without any consolidation, the number of instances s grows linearly with the total number of items n. The worst-case query complexity is therefore $O(s)$, which is equivalent to $O(n)$.*

Proof. Let n be the total number of items and let each I2CF instance have an effective capacity of C items (e.g., $C = m \cdot b \cdot \alpha$, where m is the number of buckets, b is the entries per bucket, and α is the target load factor). In an unmanaged growth model, a new instance is added every time the system has stored an additional C items. The number of instances s required to store n items is therefore $s = \lceil n/C \rceil$. As $n \to \infty$, this gives $s = \Theta(n)$. Since a query must inspect all s instances in the worst case, the query complexity is $O(s) = O(n)$. □

The CCF architecture as defined by Luo et al. explicitly includes management policies to prevent this linear degradation [18]. It is not merely a chain of filters

but a managed system that provides hierarchical scaling. This system operates at two granularities. The first is a fine-grained, bucket-level elasticity, where capacity adjustments occur within each constituent I2CF by adding or removing buckets from its consistent hash ring. The second is a coarse-grained, filter-level elasticity, which accommodates sudden load spikes by adding entire new I2CF instances. To control the growth of query latency, the architecture incorporates a consolidation mechanism, which periodically merges underutilized instances to reduce the total number of instances, s. This hierarchical approach allows the system to manage capacity at multiple granularities, handling gradual growth through the fine-grained mechanism and absorbing sudden traffic spikes by adding new instances, while maintaining long-term performance through background consolidation.

2.3.6 Concurrency Control Strategies

The deployment of Cuckoo Filters in a distributed environment introduces concurrency hazards. During an insertion operation, the filter performs a sequence of displacements that modify multiple bucket locations. A concurrent query examining these locations during this transient state may fail to find an element that has been evicted from its original bucket but not yet placed in its alternate location. This race condition, which we term a *false miss*, results in a definitive negative answer for an element present in the set, thereby violating the data structure's membership guarantee.

We analyze three distinct concurrency control strategies, each designed to resolve the false-miss problem for a specific system architecture and workload pattern.

The first strategy, optimistic concurrency, is designed for read-dominant workloads, such as those found in caching systems. Fan et al. developed this model for the MemC3 system [20]. The model serializes write operations while allowing multiple readers to proceed without locks, using version counters for synchronization. A key component of this model is the execution order of relocations along a displacement chain. After identifying a valid path, the system moves the last element in the chain into the empty slot first, which frees a new slot. The second-to-last element is then moved into this newly vacated slot. This reverse-order execution ensures that every element being relocated remains accessible in at least one of its two candidate buckets at all times.

Definition 2.20 (Optimistic Cuckoo Hashing [20])**.** A writer maintains a version counter for each bucket. Before initiating relocations, it identifies a valid displacement path (e.g., $a \to b \to c \to$ empty) without acquiring locks. The relocations are then executed in reverse order, with each move incrementing the version counter of the involved bucket. A reader records the versions of its two candidate buckets before inspection and rechecks them afterward. If any version has changed, the reader discards its result and retries the operation.

This strategy is highly effective for workloads where the read-to-write ratio is high, as readers do not incur lock acquisition overhead.

For general-purpose multi-core systems with more balanced read-write workloads, we analyze a fine-grained locking model that enables greater writer parallelism. The libcuckoo library[1] implements this model using lock striping, where an array of independent locks protects disjoint sets of buckets [21, 22]. A thread acquires locks only for the buckets involved in its operation, which permits concurrent write operations on non-overlapping regions of the filter.

For disaggregated memory architectures that utilize Remote Direct Memory Access (RDMA), in-memory locking mechanisms are inapplicable. Clients access remote memory directly over the network. Grant et al. developed RCuckoo for this environment, which employs a lease-based locking protocol [23]. In this model, clients acquire time-bounded leases on remote buckets via atomic RDMA operations. The automatic expiration of these leases provides fault tolerance to client failures without requiring explicit recovery protocols.

Definition 2.21 (Lease-Based Locking for RDMA [23])**.** For disaggregated memory architectures, we manage client failures during write operations through a lease-based protocol. The system maintains a separate lease table in remote memory, which is distinct from the primary lock table for normal operations. A client identifies a potentially stranded lock when an operation exceeds a predefined timeout threshold. To initiate recovery, the client acquires an exclusive repair lease for the affected index region via an atomic compare-and-swap operation on the lease table. This operation grants the client exclusive rights to modify the table state for recovery purposes. Upon completion of

[1] `https://github.com/efficient/libcuckoo`

the repair, the client relinquishes the lease by clearing its ownership flag. The repair lease itself is subject to a timeout mechanism, which provides fault tolerance against clients that fail during the recovery process itself.

We note that across all three concurrency models, the no-false-negatives property of the Cuckoo Filter remains invariant. A query that returns false provides a correct answer: the element is not, and has never been (unless subsequently deleted), a member of the set. The specific concurrency strategy is an implementation detail dictated by the hardware topology and workload, but the membership guarantee is preserved.

2.3.7 Replication and Convergence

When we replicate Cuckoo Filters across multiple nodes for fault tolerance, the replicas may temporarily diverge due to network latency in propagating updates. We analyze the consequences of this divergence, particularly for deletion operations, and compare the behavior of Cuckoo Filters to that of standard and Counting Bloom Filters.

A standard Bloom filter provides no mechanism for deletion; elements, once inserted, remain in the filter indefinitely. A Counting Bloom Filter supports deletion by decrementing counters but at the cost of an approximately four-fold increase in space overhead [14]. A Cuckoo Filter, by contrast, implements deletion as a direct removal of the element's fingerprint.

In a replicated environment, this distinction becomes significant. Consider a Cuckoo Filter replicated on nodes n_1 and n_2. An insertion of element x on n_1 propagates asynchronously to n_2. During the propagation window, a query for x on n_2 will return false. This behavior, a form of temporal false negative, is inherent to any asynchronously replicated system.

The key difference emerges in deletion scenarios, a problem central to systems that use tombstones to mark deleted data under eventual consistency [10]. When element x is deleted on n_1, its fingerprint is immediately removed from the local filter. During the propagation window, a query for x to n_1 correctly returns false, while a query to n_2 returns a stale positive. Once the deletion operation propagates to n_2, its fingerprint is also removed, and both nodes converge to a consistent state where x is correctly identified as absent.

A Bloom filter behaves differently. Since it cannot remove the element, it will continue to return positive for the deleted key indefinitely. This forces the system to perform expensive validation lookups against the backing data store merely to discover a tombstone, negating much of the filter's performance benefit in workloads with frequent deletions [10].

Proposition 2.22 (Cuckoo Filter Replication Convergence)**.** *In an eventually consistent system with replicated Cuckoo Filters, let an element x be inserted at time t_i and deleted at time $t_d > t_i$. For any replica r, there exist propagation delays such that the insertion is applied at time $\tau_i \geq t_i$ and the deletion is applied at time $\tau_d \geq t_d$. For all times $t > \tau_d$, a query for x on replica r will deterministically return false. In contrast, a standard Bloom filter provides no upper bound on the time until a query for a deleted element ceases to return a positive result.*

This property enables Cuckoo Filters to converge more rapidly to the true state of the system, especially in environments with frequent deletions. For distributed systems implementing eventual consistency, this allows the filter's state to accurately track operations such as the purging of tombstones during compaction or garbage collection. The ability to remove fingerprints, rather than merely marking them for future cleanup, ensures that the filter remains a reliable oracle for the live, non-deleted state of the dataset.

2.3.8 Impact on Distributed System Architectures

We analyze the concrete architectural impact of Cuckoo Filters in several domains where distributed systems require both efficient membership testing and dynamic element deletion.

Our first application domain is Log-Structured Merge-Tree (LSM-tree) based key-value stores. In these systems, data is organized into immutable, sorted files on disk (SSTables), each typically indexed by an in-memory filter. Deletions are often handled by writing a tombstone marker rather than by immediate data removal [10]. A standard Bloom filter, being unable to remove entries, continues to return a positive match for a deleted key. This behavior forces the database to perform a disk I/O operation only to discover the tombstone, incurring unnecessary latency. A Cuckoo Filter resolves this inefficiency. During data compaction, the fingerprint corresponding to a purged tombstone can be removed from the filter. Subsequent queries

for the deleted key will correctly return false, eliminating the wasted I/O. The Chucky architecture further exploits this by unifying the multiple per-level filters of an LSM-tree into a single Cuckoo Filter, which reduces the point query complexity from O(number of levels) to $O(1)$ [24].

In named-data networking (NDN), we examine the Pending Interest Table (PIT), which tracks outstanding content requests. The DiCuPIT system [25] utilizes distributed Cuckoo Filters to replace traditional Bloom filter-based implementations. This architectural change yields a 36% reduction in lookup time and a 31% reduction in memory consumption compared to Bloom filter baselines. The primary advantage is the ability to correctly and efficiently delete PIT entries as their corresponding data responses arrive.

Our third domain is state synchronization in blockchain networks. Nodes in these systems maintain a mempool of pending transactions. To propagate a new block efficiently, the Gauze protocol employs Cuckoo Filters to summarize the contents of a node's mempool [26]. A node propagating a block sends this compact filter to its peers. The receiving nodes use the filter to identify which transactions in the new block are absent from their local mempool and request only this delta. This method reduces block propagation bandwidth by 40 to 50%. The efficiency gain is a direct result of the filter's compact representation; a filter for a million-element set requires only a few megabytes of space.

2.4 Mutability-Composability Tradeoff

In 2.2 we saw how that the standard Bloom filter lacks dynamic mutability: it cannot delete elements without introducing false negatives or incurring substantial space overhead through auxiliary counters. The variants presented (2.6, 2.7) each resolve one constraint while introducing another. The Counting Bloom Filter provides deletion at a four-fold space cost. The Scalable Bloom Filter provides capacity growth but sacrifices the properties required for distributed aggregation.

We now examine distributed deployment patterns developed specifically for Bloom filters. These patterns address challenges that emerge when filters are replicated or partitioned across nodes: temporal inconsistency, heterogeneous filter parameters, and the need for state aggregation. We analyze two representative architectures and classify them according to the framework properties they provide.

2.4.1 Parallel Partitioned Bloom Filter

The Scalable Bloom Filter (2.7), accommodates unbounded growth through a sequence of filters with heterogeneous parameters. This heterogeneity prevents bitwise operations such as OR for set union. Two SBF instances cannot be merged because their constituent filters differ in size and hash function count. This absence of composability renders the SBF unsuitable for distributed aggregation tasks where partial results from different nodes must be combined.

The Parallel Partitioned Bloom Filter (ParBF) addresses this limitation by constructing a homogeneous sequence of sub-filters [6]. Each sub-filter in the sequence shares the same size m and the same number of hash functions k.

Definition 2.23 (Parallel Partitioned Bloom Filter [6])**.** A ParBF is a sequence of s homogeneous Bloom filters, $B_1, B_2, \ldots, B_s$. Each filter B_i has size m and uses the same k hash functions. New elements are inserted into the filter with the lowest current load. When all filters exceed a threshold load factor, a new filter B_{s+1} with the same parameters (m, k) is appended to the sequence.

The homogeneity of the constituent filters enables bitwise union operations. Two ParBF instances representing sets S_1 and S_2 can be merged by computing the bitwise OR of corresponding sub-filters: the i-th sub-filter of the merged ParBF is the OR of the i-th sub-filters from both inputs. This operation produces a ParBF representing the set union $S_1 \cup S_2$. The result satisfies the one-sided error property: queries for elements in $S_1 \cup S_2$ return true with probability 1, and queries for elements not in the union return false positive with a probability bounded by the false positive rates of the constituent filters.

The architecture further partitions the sequence of sub-filters into groups to enable parallel query execution. A query operation can be distributed across multiple threads, each inspecting a disjoint subset of the filter sequence.

This structure provides a form of composability[2]: the bitwise OR operation is commutative, associative, and

[2] In Section 4 we will expand this concept

idempotent. Two ParBF instances can be merged without coordination regarding the order of operations. However, the ParBF does not satisfy the conditions of dynamic mutability as defined in Definition 2.8. The underlying Bloom filters cannot delete elements. Once an element is inserted into any sub-filter in the sequence, its contribution to the bit array is permanent. A delete operation would require either clearing bits (introducing false negatives) or maintaining auxiliary counters (violating the space complexity constraint).

The ParBF thus represents an architectural solution to the composability limitations of the Scalable Bloom Filter but inherits the fundamental constraint of the standard Bloom filter: the inability to synchronize its state with a dataset that undergoes deletion.

2.4.2 Distributed Bloom Filter Protocol

In a replicated system where Bloom filters are maintained on multiple nodes, asynchronous propagation of updates introduces temporal inconsistency. When an element is inserted on node n_1 and the update has not yet propagated to replica n_2, a query for that element on n_2 will return false. This behavior constitutes a temporal false negative: from the perspective of the global system state, the element is present, yet the oracle returns a definitive negative.

The Distributed Bloom Filter (DBF) protocol [27], addresses this inconsistency through a probabilistic convergence mechanism. The protocol does not attempt to eliminate temporal false negatives during the propagation window. Instead, it structures peer-to-peer interactions to guarantee convergence over multiple synchronization rounds while minimizing bandwidth consumption.

The protocol employs unique, pair-wise hash functions for each interaction between nodes. This design ensures that hash collisions are statistically independent across successive synchronization rounds. Each node pair derives a unique hash function family from a seed computed as the XOR of their node identifiers.

Definition 2.24 (Distributed Bloom Filter [27])**.** Let two nodes, n_i and n_j, with unique identifiers ID_i and ID_j, wish to reconcile their local sets. The DBF protocol employs a unique, pair-wise hash function family $\mathscr{H}_{ij}$ generated from a seed s_{ij}, typically computed as $s_{ij} = ID_i \oplus ID_j$. A set of k intermediate hashes $\{h_1^{ij}, \ldots, h_k^{ij}\}$ is derived from this seed. For each element $x \in \mathscr{U}$ with a pre-computed hash $H(x)$, the final hash values are computed as:

$$H_l^{ij}(x) = (H(x) \oplus h_l^{ij}) \pmod m \quad \forall l \in \{1, \ldots, k\}$$

This method allows for the computationally inexpensive generation of unique Bloom filters for each peer interaction.

The protocol deliberately utilizes filters with a high individual false positive rate (e.g., 50%) to minimize bandwidth consumption. While a single exchange is highly probabilistic, the use of unique mappings for each interaction ensures that true set differences are revealed consistently, while random errors from false positives average out over multiple interactions. This approach sacrifices single-shot accuracy for rapid long-term convergence, a trade-off well-suited for eventually consistent systems [27]. The system-wide error rate diminishes exponentially with the number of participating nodes.

Proposition 2.25 (DBF System-Wide Error Rate [27])**.** *Let a system of N nodes use the DBF protocol, where each individual filter has a false positive rate ε. For an element $y \notin \bigcup_{i=1}^{N} S_i$, the probability that a query for y from a specific node to all of its $N-1$ peers results in a system-wide false positive is $\varepsilon_{System} \leq \varepsilon^{N-1}$.*

Proof. Let $\varepsilon = (1 - e^{-kn/m})^k$ be the false positive rate for a single filter. A system-wide false positive for a query from node n_j regarding an element y occurs if all $N-1$ peers it communicates with independently return a false positive. Let E_i be the event of a false positive from peer n_i. The DBF protocol's use of unique seeds s_{ji} for each peer interaction ensures that the hash function families $\mathscr{H}_{ji}$ are statistically independent for each peer i. Consequently, the events E_i are independent. The probability of a joint failure is the product of their individual probabilities:

$$P(\bigcap_{i=1}^{N-1} E_i) = \prod_{i=1}^{N-1} P(E_i) = \varepsilon^{N-1}$$

□

The protocol addresses temporal inconsistency through probabilistic convergence but does not provide dynamic mutability. The underlying Bloom filters cannot delete elements; once an element is inserted during a synchronization round, it remains in the local filter indefinitely. If an element is deleted from the true dataset on some node, the protocol has no mechanism to propagate this deletion to peer filters.

The protocol also does not provide composability in the sense that we will define in 3.2. While the protocol uses bitwise OR operations to combine filters during synchronization rounds, the use of unique hash functions for each node pair means that filters constructed on different nodes are not directly mergeable. Two filters representing the same set but constructed with different hash seeds cannot be combined through a bitwise OR to produce a valid filter for the union. The protocol achieves eventual consistency through iterative pairwise reconciliation, not through a single, order-independent merge operation.

3 Cardinality Estimation

We now address the related but quantitatively distinct problem of estimating the number of unique elements within a large-scale dataset.

Definition 3.1 (The Cardinality Estimation Problem)**.** Given a multiset S of elements drawn from a universe $\mathscr{U}$, the cardinality estimation problem is to determine $|S|$, defined as the number of distinct elements in S.

In a distributed, data-parallel framework, a naive implementation of this task necessitates a multi-stage execution. In an initial stage, each parallel worker processes a partition of the input multiset to produce a local set of unique elements. Subsequently, the framework must perform a shuffle operation, transmitting all unique elements from all partitions across the network to a common set of reducer nodes for final aggregation and deduplication [2].

This shuffle operation constitutes the primary bottleneck. The communication cost of this approach is not fixed; it is proportional to the number of unique elements and their size. For datasets with high cardinality, the network I/O required for the shuffle dominates the total execution time, rendering the approach impractical for interactive or large-scale analytics.

The challenge here is to enable cardinality aggregation without the shuffle of raw data. A distributed cardinality estimation oracle must support the combination of partial estimates computed on different data partitions into a global estimate, where this combination operation satisfies three requirements.

First, produces a result with error bounds identical to those of an oracle constructed in a single pass over the complete dataset. This ensures that distributed aggregation does not introduce additional approximation error beyond the inherent error of the probabilistic estimator.

Second, executes in time independent of the dataset cardinality. The combination of two partial estimates must require computational cost proportional only to the size of the estimate representations, not to the number of elements they summarize. This transforms the communication-bound shuffle operation into a fixed-cost sketch transmission.

Third, can be performed in any order without coordination between nodes. The final global estimate must be identical regardless of the sequence in which partial estimates are combined. This enables distributed reduction without sequential dependencies or central coordination bottlenecks.

We observe that these three requirements collectively specify a class of binary operations over cardinality estimation oracles. The requirement for a property that satisfies these conditions motivates the subsequent paragraphs.

3.1 Composability

The constraints identified in the preceding sections impose requirements on the structure of a distributed cardinality estimation oracle. We now introduce the property that resolves the shuffle bottleneck constraint.

Definition 3.2 (Composability)**.** Let $\mathscr{D}_1, \ldots, \mathscr{D}_k$ be k instances of a probabilistic oracle, where each $\mathscr{D}_i$ is constructed over a data partition S_i. The oracle exhibits composability if there exists a binary operation $\cup$ such that the combined oracle $\mathscr{D}_{agg} = \mathscr{D}_1 \cup \cdots \cup \mathscr{D}_k$ produces query results with error bounds identical to those of an oracle constructed in a single pass over the set union $S = \bigcup_{i=1}^{k} S_i$. The operation $\cup$ must be commutative, associative, and idempotent.

The three properties specified in this definition address the three architectural requirements established above. Commutativity and associativity guarantee that the final result of combining a collection of oracles is independent of the order of aggregation. A system can perform reductions in parallel, in a hierarchical tree structure, or in any arbitrary sequence without requiring coordination or ordering constraints. These properties enable the parallelizability requirement: distributed

workers can compute partial estimates concurrently, and intermediate aggregators can combine these estimates in any order dictated by network topology or load balancing considerations.

Idempotency ensures that re-combining a duplicate oracle does not alter the aggregate state. This simplifies fault-tolerance protocols. A distributed aggregation system can employ message delivery with at-least-once semantics, obviating the need for more complex exactly-once delivery mechanisms. If a network failure causes a partial estimate to be retransmitted and merged multiple times, the idempotency property guarantees that the final result remains correct.

Collectively, these properties enable the non-additive `COUNT(DISTINCT)` problem to be reframed as a decomposable aggregation problem, analogous to a `SUM` operation. The communication cost of distributed cardinality estimation becomes proportional to the size of the compact oracle representation, not to the dataset cardinality. This transformation is the architectural consequence of composability.

3.2 HyperLogLog

The HyperLogLog sketch satisfies the conditions established in 3.2. Its union operation executes as a register-wise maximum, requiring time proportional only to the number of registers m, independent of dataset cardinality. This operation is commutative, associative, and idempotent. The property derives from the design of the sketch as a fixed-size summary that encodes cardinality information through the maximum observable in each of m independent stochastic experiments.

3.2.1 Stochastic Averaging and the Harmonic Mean

We sum up the core mechanics of the HyperLogLog algorithm as introduced by Flajolet et al. [4]. The algorithm maps elements to an array of registers and records an observable based on the bit patterns of their hash values.

Definition 3.3 (HyperLogLog Sketch)**.** A HyperLogLog sketch consists of an array M of $m = 2^p$ registers, where p is a precision parameter. The sketch is associated with a hash function $h : \mathcal{U} \to \{0,1\}^L$ that maps elements from a universe $\mathcal{U}$ to binary strings of sufficient length L. Each register $M[j]$ is initialized to 0.

The algorithm processes a multiset by updating this array of registers. For each element, a single hash value is used to select a register and compute an update value. This process, known as stochastic averaging, uses one hash function to simulate m independent experiments, with each register representing one experiment.

To insert an element $v \in S$, we first compute its hash value $x = h(v)$. The first p bits of x are used to determine a register index $j \in \{0,\dots,m-1\}$. Let w be the remaining $L-p$ bits of x. We then compute an observable $\rho(w)$, defined as one plus the number of leading zeros in w. The corresponding register is updated according to the rule $M[j] := \max(M[j], \rho(w))$. After all elements have been processed, each register $M[j]$ contains the maximum ρ value observed for any element mapped to that register.

The final cardinality estimate is derived from the state of these registers. The algorithm uses the harmonic mean of the values $2^{M[j]}$, which is less sensitive to outlier values than the arithmetic or geometric mean.

Proposition 3.4 (HyperLogLog Estimator [4])**.** *Given a HyperLogLog sketch with m registers M, the cardinality estimate E is computed as:*

$$E = \alpha_m m^2 \left(\sum_{j=0}^{m-1} 2^{-M[j]} \right)^{-1}$$

where α_m is a bias correction constant that depends on m.

This mechanism produces a fixed-size sketch of $\Theta(m \log\log n)$ bits that summarizes the cardinality of a set containing n elements. The theoretical relative error of this estimate is approximately $1.04/\sqrt{m}$ [4].

3.2.2 Union and Algebraic Properties

The utility of the HyperLogLog sketch in distributed systems derives from its support for an efficient union operation. This operation allows sketches computed on disparate data partitions to be combined to produce a sketch representing the union of the underlying sets.

Definition 3.5 (HyperLogLog Union [5])**.** Let M_1 and M_2 be two HyperLogLog sketches of m registers, representing sets S_1 and S_2 respectively. The union sketch, $M_{1\cup 2}$, representing the set $S_1 \cup S_2$, is computed as the register-wise maximum of the two input sketches:

$$M_{1\cup 2}[j] := \max(M_1[j], M_2[j]) \quad \forall\, j \in \{0,\dots,m-1\}$$

The resulting sketch $M_{1\cup2}$ is identical to the sketch that would have been generated by processing all elements from both S_1 and S_2 in a single pass.

Theorem 3.6 (Properties of the Union Operation [5])**.** *The HyperLogLog union operation is commutative, associative, and idempotent. For any sketches M_A, M_B, and M_C we have commutativity*

$$M_A \cup M_B = M_B \cup M_A$$

associativity

$$(M_A \cup M_B) \cup M_C = M_A \cup (M_B \cup M_C)$$

and idempotency

$$M_A \cup M_A = M_A$$

The register-wise maximum operation constitutes the binary operation $\cup$ required by 3.2. The commutativity and associativity guarantee order-independent aggregation. The idempotency ensures that the operation satisfies the requirements for fault-tolerant distributed reduction.

Commutativity and associativity guarantee that the final result of merging a collection of sketches is independent of the order of aggregation. This allows a system to perform reductions in parallel, in a hierarchical tree structure, or in any arbitrary sequence without requiring coordination or ordering constraints. Idempotency ensures that re-merging a duplicate sketch does not alter the aggregate state. This simplifies fault-tolerance protocols, as it allows for message delivery with at-least-once semantics, obviating the need for more complex exactly-once delivery mechanisms for sketch aggregation [5].

Collectively, these properties enable the non-additive `COUNT(DISTINCT)` problem to be reframed as a decomposable aggregation problem, analogous to a `SUM`. This allows the computation to be parallelized with minimal communication overhead [2].

3.3 Architectural Implications

These properties, however, describe the logical behavior of sketch composition, not the physical realities of executing such compositions over millions of instances in a resource-constrained environment. When applied to common analytical workloads, such as large-scale group-by aggregations, the standard HLL algorithm encounters limitations related to estimation accuracy at low cardinalities and communication overhead not apparent from its initial formulation.

3.3.1 Challenges at Scale

We analyze two primary challenges that arise when deploying the standard HyperLogLog algorithm in a data-parallel execution environment. Both are consequences of the data distribution patterns common in large-scale analytical queries.

First, analytical workloads frequently involve a high degree of cardinality skew. In a distributed `GROUP BY` operation executed over billions of records, the resulting groups often follow a power-law distribution. A small number of groups may have very high cardinalities, but the vast majority of groups will contain very few unique elements [5]. The original HLL estimator exhibits significant systematic bias and a larger relative error for these low-cardinality sets, which compromises the accuracy of the estimates for the long tail of the distribution.

Second, the aggregation of a massive number of sketches introduces a new form of communication overhead. In a distributed `GROUP BY` query, the map phase generates a separate HLL sketch for each group on each worker node. While each individual sketch is small, the total number of sketches can be in the millions or billions. The subsequent shuffle phase must serialize, transmit, and deserialize this large volume of small objects. This process can itself become a network and CPU bottleneck, undermining the efficiency gains achieved by avoiding the shuffle of raw data [5].

3.3.2 HyperLogLog++

The HyperLogLog++ algorithm introduces a series of engineering refinements to the standard HLL to address the challenges of accuracy and communication overhead at scale [5].

First, to address the systematic bias at low cardinalities, it replaces the small-range correction with a bias correction mechanism derived from empirical data. The system uses a pre-computed table of bias values obtained from simulations to correct the raw estimate, improving accuracy for the low-cardinality groups prevalent in skewed analytical workloads [5].

Second, it forces the use of a 64-bit hash function. This change expands the hash space to make collisions statistically insignificant even for cardinalities exceeding billions, thereby eliminating the need for the large-range correction formula of the original algorithm and simplifying the implementation [5].

The most significant architectural refinement is the introduction of a dual-representation sketch that adapts its storage format based on cardinality. This design directly addresses the communication overhead associated with shuffling a massive number of sketches.

Definition 3.7 (Sparse and Dense Representations [5])**.** A HyperLogLog++ sketch can exist in one of two formats. A `dense` format, which is the standard HLL representation: a fixed-size array of m registers. A `sparse` format, used for low cardinalities, which stores only the non-zero registers as a compressed, sorted list of (index, value) pairs. The sketch begins in the `sparse` format and transitions to the `dense` format only when its size in the sparse representation would exceed the size of the dense format.

The sparse format reduces the memory footprint of these numerous low-cardinality sketches by an order of magnitude or more. This reduction in size directly translates to a commensurate reduction in the volume of data that must be serialized, transmitted during the shuffle phase, and deserialized by reducer nodes [5].

3.4 Architectures and Implementations

The properties of HyperLogLog sketches, particularly their efficient and composable union operation, enable their integration into diverse distributed computing architectures. The choice of architecture is typically dictated by the nature of the data workload, whether batch or streaming, and the desired query patterns.

3.4.1 Architectural Patterns

We analyze three canonical patterns for distributed HLL computation. The first is the MapReduce paradigm. In this model, an input dataset is partitioned across worker nodes. The map stage generates local HLL sketches for each data partition, creating a separate sketch for each grouping key in a group-by operation. The system then shuffles these compact sketches to reducer nodes. In the reduce stage, each reducer applies the union operation to merge the sketches it receives for a given key. The correctness of this stage, where sketches may arrive in an arbitrary order, is guaranteed by the commutativity and associativity of the union operation [2].

The second pattern is designed for real-time aggregation in stream processing systems. This architecture partitions an unbounded data stream into logical windows, such as fixed time intervals or element counts. For each active window, an operator maintains a local HLL sketch. The composability of sketches allows for efficient computation over combined windows. To compute the cardinality over a time interval T composed of sub-intervals $t_1, \ldots, t_k$, the system retrieves the corresponding sketches $M_{t_1}, \ldots, M_{t_k}$ and computes the final sketch as $M_T = \bigcup_{i=1}^{k} M_{t_i}$. The cost of this operation is independent of the raw data volume within the interval, enabling efficient, stateful stream processing [28].

The third pattern is a hierarchical, or tree-based, aggregation model, well-suited for systems with a natural data hierarchy [29]. In this model, leaf nodes generate sketches from raw data streams. Intermediate nodes in the hierarchy periodically collect and merge the sketches from their children to create higher-level aggregates. The validity of this hierarchical model rests upon the associativity of the HLL union, which ensures that the final global sketch is identical regardless of the tree's depth or the order of intermediate merge operations. This pattern enables multi-resolution querying, where queries for cardinality at different levels of the hierarchy can be answered by accessing pre-aggregated sketches without reprocessing data from lower levels.

3.4.2 Implementation Case Studies

The patterns enabled by HyperLogLog are realized in a spectrum of production systems, ranging from low-level data structures requiring explicit user orchestration to high-level, transparent query optimizations managed entirely by a database engine. We analyze three representative implementations.

The Redis implementation [30] provides HLL as a primitive data type. It exposes three core commands: `PFADD` to insert elements, `PFCOUNT` to retrieve the cardinality estimate, and `PFMERGE` to perform the union of multiple sketches. An HLL sketch is represented as a fixed-size string. This facilitates a client-driven model for distributed computation: an application can retrieve

a serialized sketch from one node, transmit it over the network, and merge it with another sketch on a different node by invoking `PFMERGE`. In this model, the orchestration of the distributed aggregation is the explicit responsibility of the application logic.

In contrast, distributed SQL query engines such as Presto and Google BigQuery integrate HLL as a high-level, transparent optimization within the query execution layer. These systems expose the functionality through an aggregate function, typically `APPROX_DISTINCT`. When executing a distributed query, the query planner automatically pushes the sketch creation stage to the worker nodes. The workers compute HLL++ sketches, often utilizing both sparse and dense representations to optimize for memory and network usage. In the final stage, these partial sketches are shuffled to a reducer or coordinator node, which performs a final merge aggregation. The entire process is managed by the query optimizer, abstracting the underlying mechanics from the user [5]. BigQuery further provides a multi-stage API (`HLL_COUNT.INIT`, `HLL_COUNT.MERGE`) that allows intermediate, partially aggregated sketches to be materialized in tables, enabling efficient incremental rollups [31].

The highest level of abstraction is represented by systems like the Citus extension for PostgreSQL. This system provides transparent distributed cardinality estimation through automatic query rewriting. An administrator can enable approximate counting via a configuration parameter. Subsequently, the distributed query planner automatically rewrites standard `COUNT(DISTINCT)` queries to use an HLL-based implementation. It pushes sketch aggregation down to the worker nodes and merges the intermediate sketches on the coordinator node to produce the final result. From the user's perspective, the use of HLL is a transparent physical-layer optimization managed entirely by the database [1].

3.5 Challenges in Production Environments

The deployment of probabilistic data structures in production systems introduces several difficulties. We will analyze three such challenges: ensuring fault tolerance efficiently, providing security against adversarial manipulation, and managing privacy in federated contexts.

3.5.1 Fault Tolerance and Approximate Recovery

In data-parallel frameworks, fault tolerance for stateless transformations is achieved by recomputing lost partitions from their lineage graph [2]. For stateful operations, such as maintaining HLL sketches over streaming data, this approach is insufficient. Traditional fault tolerance for stateful operators requires periodic checkpointing of the entire state, which incurs significant I/O overhead and compromises performance [28].

An alternative approach, termed approximate fault tolerance, leverages the inherently probabilistic nature of the data structure to create more efficient recovery mechanisms. Instead of checkpointing at fixed intervals, this model triggers a backup only when the potential error introduced by a failure would exceed a user-defined threshold. This is managed by monitoring the divergence between the current in-memory state and its last persisted backup [28].

Definition 3.8 (State Divergence [28])**.** Let $M_{current}$ be the current state of an HLL sketch and M_{backup} be its most recent persisted state. The state divergence, $D(M_{current}, M_{backup})$, is a metric quantifying the difference between the two sketches.

The AF-Stream (Approximate Fault-Tolerant Stream) framework defines this divergence based on the specific properties of the streaming algorithm being used. For HLL, this can be modeled as a function of the differences between the register values.

It executes a backup operation only when the state divergence D exceeds a pre-configured threshold Θ, or when the number of unprocessed items since the last backup exceeds another threshold L [28].

Proposition 3.9 (Bounded Error Under Failures [28])**.** *In the AF-Stream model, let a worker experience k failures. The total divergence between the actual state and the ideal, failure-free state is bounded by a function of the user-specified thresholds Θ and L, and algorithm-specific constants. The error bound is independent of the number of failures k.*

3.5.2 Security Against Adversarial Manipulation

The probabilistic nature of HyperLogLog, while providing efficiency, also introduces security vulnerabilities. We analyze a specific class of adversarial action,

the cardinality inflation attack, where a malicious actor seeks to deliberately produce an arbitrarily large cardinality estimate from a small set of crafted inputs [32].

The attack leverages the internal structure of the HLL algorithm. An attacker can empirically discover input elements that produce high ρ values for each of the m registers. Even without knowledge of the specific hash function, an attacker can construct a malicious set by repeatedly generating random inputs and observing their effect on a local HLL instance. By collecting just m such inputs, one targeting each register, an attacker can construct a set that populates every register with a high value. When this small set is inserted into the target HLL sketch, the harmonic mean calculation yields a massive, artificially inflated cardinality estimate. This vulnerability has been demonstrated to be effective against production implementations in systems such as Presto and Redis [32].

We consider two classes of mitigation strategies. The first involves cryptographic techniques, such as applying a secret, system-specific salt to the hash function. This approach makes it computationally difficult for an external attacker to craft malicious inputs offline. However, this strategy breaks the mergeability property for sketches that use different salts, limiting its applicability in systems that rely on sketch union for aggregation across different administrative domains [32].

The second class of strategies involves an aggregation mechanisms designed to operate in the presence of Byzantine adversaries. Instead of a simple `max` operation during a merge, or a simple harmonic mean for estimation, the system can employ aggregation rules that are resilient to outliers. Techniques from the field of Byzantine-resilient distributed optimization, such as the Geometric Median or aggregation rules like Krum, are designed to filter out malicious updates [33]. The Geometric Median, for instance, finds a point that minimizes the sum of Euclidean distances to a set of input vectors, making it inherently robust to arbitrarily corrupted inputs. Applying such aggregators to the vector of HLL registers during a merge operation is an active area of research for building secure, Byzantine-fault-tolerant analytical systems [34].

3.5.3 Privacy Implications in Federated Systems

While HLL sketches provide a compact summary that does not store raw data elements, their use in federated environments introduces privacy considerations. The act of sharing a sketch, even if it contains only anonymized hash-derived values, can still leak information about the underlying dataset, particularly concerning unique or rare elements [33].

In a federated query system, an adversary who compromises the central aggregator or observes the transmitted sketches could perform linkage attacks. If a hospital finds that only one patient satisfies the criteria for a query, sharing an HLL sketch of this single-element set still reveals information. An attacker could potentially infer properties of this unique patient by correlating the sketch with a known background population [33]. To quantify this risk, we can apply the concept of k-anonymity to the buckets of an HLL sketch.

Definition 3.10 (Bucket k-Anonymity [33])**.** Let B be a set of elements matching a query, drawn from a background population A. An HLL bucket is defined to be k-anonymous if the value it contains could have been generated by at least k distinct elements from the background population A. An attacker who observes the sketch cannot determine with certainty which of the k elements contributed to that bucket's value.

The level of k-anonymity provided by an HLL sketch is a function of the sketch parameters, the size of the background population, and the size of the query set. Research has shown that it is possible to compute theoretical bounds on the expected number of buckets in a sketch that are not k-anonymous. HLL sketches are particularly effective for rare diseases (where the ratio of query set size to background population size is low). This is in marked contrast to sharing exact counts, where a count of one immediately breaks anonymity [33]. This allows us to manage the trade-off between the accuracy of the cardinality estimate, which increases with the number of buckets m, and the privacy risk, as a larger m may decrease the expected k-anonymity of each bucket.

3.6 Limitations and Variants

The value of a probabilistic data structure is determined not only by its space-error characteristics but also by the set of operations it supports. The HyperLogLog algorithm is highly specialized for cardinality estimation and set union. While its merge operation is fundamental for distributed aggregation, it does not natively support other operations such as intersection or set difference.

Alternative data structures, such as MinHash and its derivatives like Theta Sketches, provide a richer set of supported operations. These sketches are designed to estimate the Jaccard similarity between sets, from which the sizes of their union, intersection, and difference can be derived. However, this increased functionality comes at a cost. For the primary task of cardinality estimation, HLL is substantially more space-efficient than MinHash-based sketches providing the same estimation accuracy [35].

3.6.1 Variants for Cardinality Estimation

The UltraLogLog algorithm modifies the underlying register structure to encode more information per bit [36]. It generalizes the HyperLogLog register by partitioning its bits. A set of q bits stores the maximum update value u_i seen so far, while a set of d additional bits acts as a compact summary, indicating whether updates with values $u_i - 1, \ldots, u_i - d$ have occurred. This design, particularly for the ULL configuration with $q = 6$ and $d = 2$, fits into a single 8-bit register. It achieves a theoretical memory-variance product approximately 28% lower than a standard 6-bit HLL sketch while remaining commutative, idempotent, and mergeable.

The HyperLogLogLog algorithm[3] achieves compression through a sparse-dense architecture based on value offsets [37].

Definition 3.11 (HyperLogLogLog Sketch Structure [37])**.** A HyperLogLogLog sketch is a 3-tuple (S, M, B) where B is a base value, representing a lower bound for the majority of register values.

M is a dense array storing small-width integer offsets for registers whose values are close to B. For a register j where $B \leq M'[j] < B + 2^k$, its offset $M[j] = M'[j] - B$ is stored using k bits.

S is a sparse associative array storing pairs $(j, M'[j])$ for outlier registers whose values fall outside the dense offset range. S is a sparse associative array storing pairs $(j, M'[j])$ for outlier registers whose values fall outside the dense offset range.

The base value B is chosen dynamically to minimize the total representation size $k \cdot m + |S| \cdot (\log m + \log w)$.

This structure adapts to the statistical distribution of register values, using compact offsets for the common case and a sparse map for exceptions. The resulting sketch requires approximately $m \log_2 \log_2 \log_2 m$ bits for large n, albeit with a potential increase in the computational complexity of updates [37].

Several open problems remain at the intersection of cardinality estimation and distributed systems. The development of robust aggregation protocols that are resilient to Byzantine adversaries is an active area of research. Securing sketch-based systems requires moving beyond simple merge logic to robust aggregation rules that can tolerate corrupted inputs. Furthermore, integrating formal privacy guarantees with cardinality estimation sketches is necessary for enabling collaborative analytics on sensitive data. Finally, new architectural paradigms such as sketch disaggregation, where a single logical sketch is fragmented and distributed across network hardware, present a frontier for research into ultra-low-latency, in-network analytics [38].

4 A Framework for Architectural Deployment

In the two previous sections we derived two properties from distributed deployment constraints. Section 2 established dynamic mutability (Definition 2.8) as the requirement for state synchronization in systems where datasets undergo deletion. Section 3 established composability (Definition 3.2) as the requirement for distributed aggregation without data shuffling. We observe that these properties address orthogonal constraints: mutability maintains oracle consistency with evolving datasets, while composability combines partial results across network-separated partitions. This section analyzes the implications of this orthogonality for structure selection and deployment.

4.1 Mutability-Composability Orthogonality

Dynamic mutability and composability address distinct requirements. A membership oracle for a dynamic dataset requires mutability regardless of whether the oracle is replicated or partitioned. A cardinality oracle for a static dataset requires composability if the dataset is partitioned, regardless of whether elements are ever deleted. The constraints impose requirements on different aspects of oracle functionality: mutability constrains the update interface, while com-

[3] Three log scientist `https://curiouscoding.nl/posts/three-log-scientist/`

posability constrains the merge interface.

The mechanisms enabling these properties impose conflicting requirements on state representation. Mutability requires retaining sufficient information to identify and remove a specific element's contribution. The Cuckoo Filter achieves this by storing explicit fingerprints in buckets at deterministically computable locations (§2.3). Deletion locates the fingerprint and erases it without affecting other entries. This explicit storage enables constant-time deletion without false negatives.

Composability requires that oracle state be derivable from commutative operations applied to element representations. Such operations necessarily lose information about individual elements. The HyperLogLog sketch (3.3) achieves this by retaining only maximum observables in each register, discarding element identities (subsection 3.2). The register-wise maximum operation used for sketch union cannot distinguish which specific elements contributed to a register's current value. This information loss enables order-independent aggregation.

A structure that retains sufficient information for targeted deletion cannot perform order-independent aggregation, as the merge operation must track which elements have been processed to avoid double-counting. Conversely, a structure that performs order-independent aggregation through lossy compression cannot support targeted deletion, as the information required to identify a specific element's contribution has been discarded. This reflects a trade-off between maintaining element-level granularity and achieving compression through information loss.

4.1.1 Structural Incompatibility

We establish that idempotent merge operations preclude the information retention required for deletion without false negatives.

Definition 4.1 (Reversible Element Representation)**.** An oracle $\mathscr{D}$ maintains a reversible representation of element x if there exists a deterministic procedure $\textsc{Locate}(x,\mathscr{D})$ that identifies the complete contribution of x to the oracle's state, and a corresponding operation $\textsc{Remove}(x,\mathscr{D})$ that eliminates this contribution without affecting the query correctness for any other element $y \neq x$.

The Cuckoo Filter satisfies this definition. The $\textsc{Locate}$ procedure computes the two candidate bucket indices from x and searches for the fingerprint $f = \textsc{fingerprint}(x)$ in those buckets. The $\textsc{Remove}$ procedure erases one copy of f from the bucket where it is found. The fingerprint location is deterministically computable from x, and removing it does not alter other stored fingerprints.

The HyperLogLog sketch does not satisfy this definition. An element x contributes the observable $\rho(w)$ to register $M[j]$ through the update rule $M[j] := \max(M[j], \rho(w))$. No procedure can determine, given only x and the final state M, whether $M[j]$ contains the value contributed by x or a larger value contributed by some other element $y \neq x$. The maximum operation discards the element identity.

We derive the incompatibility of idempotent composability and deletion as follows.

Theorem 4.2 (Incompatibility of Idempotent Composability and Deletion)**.** *Let $\mathscr{D}$ be a probabilistic oracle satisfying composability (3.2) with an idempotent merge operation $\cup$. If $\mathscr{D}$ supports a deletion operation* $\textsc{Delete}(x)$ *that guarantees no false negatives for elements in the represented set, then $\mathscr{D}$ must maintain reversible representations of inserted elements.*

Proof. Assume an oracle $\mathscr{D}$ satisfies idempotent merge, supports deletion without false negatives, and does not maintain reversible representations. Let $\mathscr{D}_0$ denote an empty oracle and let $\mathscr{D}_x = \textsc{Insert}(x,\mathscr{D}_0)$ denote the oracle after inserting element x. The absence of reversible representations implies that the state update produced by inserting x cannot be isolated and reversed. The contribution of x is fused with contributions from other elements through irreversible operations.

The idempotence property requires $\mathscr{D}_x \cup \mathscr{D}_x = \mathscr{D}_x$ for all states. This equality constrains the merge operation to be a projection: repeatedly applying the merge does not change the result. For this projection to be well-defined on oracles representing multisets, the merge must discard information about element multiplicity. If $\mathscr{D}_x$ represents the insertion of x once, merging it with itself cannot represent two insertions without violating idempotence. The oracle must therefore use set semantics, where each element is either present or absent, without multiplicity tracking.

The deletion operation $\textsc{Delete}(x,\mathscr{D}_x)$ must produce a state $\mathscr{D}'$ where queries for x return false with probabil-

ity 1. To achieve this without introducing false negatives for other elements, the deletion must identify and remove precisely the information that causes queries for x to return true.

Suppose the deletion removes information from the state that was contributed by elements $y \neq x$. Subsequent queries for those elements may return false despite their presence in the set, violating the one-sided error guarantee. Such behavior contradicts the requirement that deletion preserves query correctness for all elements except the deleted one.

Alternatively, suppose the deletion removes exactly the contribution of x and nothing more. This requires identifying which components of the oracle state were produced by inserting x. The assumed absence of reversible representations precludes such identification. The contribution of x is fused irreversibly with contributions from other elements, and no procedure can isolate the state components attributable solely to x without access to a reversible representation.

Both cases lead to contradiction. No oracle can simultaneously satisfy idempotent merge, deletion without false negatives, and absence of reversible representations. □

The Cuckoo Filter maintains reversible representations through explicit fingerprints in deterministically computable bucket locations. It supports deletion, but its merge operation is not idempotent. Merging two Cuckoo Filters requires checking for duplicate fingerprints to maintain correctness. The HyperLogLog sketch has an idempotent merge operation through register-wise maximum but does not maintain reversible representations. The maximum operation discards element identities, precluding deletion without oracle reconstruction from raw data.

Theorem 4.2 establishes that this dichotomy is not a consequence of algorithmic limitation but an information-theoretic necessity. Idempotent merge operations compress state through lossy operations that discard the information required for targeted deletion. Reversible representations retain element-level information that prevents lossy compression. The properties address different architectural constraints and impose incompatible requirements on state representation.

4.2 Deployment Patterns and Property Requirements

The orthogonality established in subsection 4.1 enables a classification of deployment patterns based on which constraint they address. We identify two pattern classes and derive the property requirements that enable each class.

4.2.1 Local-Execution Patterns

Distributed systems that align oracle partitioning with data partitioning enable local execution of oracle operations. A membership query for an element is routed to the node responsible for that element's partition, where both the data item and the oracle state are co-located. This eliminates network round-trips for oracle queries. The pattern applies to any workload where query routing can be determined from the queried element's hash value: membership testing in partitioned key-value stores, existence checks in distributed caches, and presence verification in content-addressable storage systems.

The viability of local-execution patterns is contingent upon oracle mutability. When data is deleted from a partition, the co-located oracle state must reflect this deletion to maintain consistency. Without deletion support, the oracle state diverges from the true partition contents. A query for a deleted element returns a false positive, forcing the system to perform a validation lookup against the backing data store. This validation lookup incurs the network round-trip cost that the oracle was designed to eliminate. As the deletion rate increases, the frequency of these wasted lookups increases proportionally, eventually negating the performance benefit of local execution.

The co-partitioning architecture (§ 2.3.4) instantiates this pattern class for membership testing. Filter buckets are assigned to the same consistent hash ring as data keys. Each node maintains both the data items and the filter buckets within its assigned token range. Theorem 2.17 establishes that expected query latency is independent of dataset size under uniform query distribution. This constant-time guarantee holds only for oracles that support deletion, as element removal must not require accessing non-local state or rebuilding global data structures.

A local-execution pattern is correct for dynamic datasets if and only if the oracle satisfies 2.8. The

Cuckoo Filter analysis in §2.3 demonstrates that deletion can be performed locally using only the element's fingerprint. Any oracle lacking deletion support accumulates stale positives proportional to the deletion rate, forcing validation lookups that violate the local-execution guarantee.

4.2.2 Hierarchical-Reduction Patterns

Distributed aggregation systems organize nodes into reduction hierarchies where intermediate nodes combine partial results from children. The pattern manifests in multiple system architectures. MapReduce implements two-level hierarchies: map workers produce partial results, reduce workers aggregate these results. Stream processing systems implement temporal hierarchies: window operators maintain sketches for time intervals, aggregator operators combine sketches across intervals. Multi-datacenter architectures implement spatial hierarchies: regional aggregators combine results from local data centers, global aggregators combine regional results.

The correctness of hierarchical reduction is contingent upon merge operation commutativity, associativity, and idempotency. Commutativity ensures that intermediate nodes can merge results from children in any order. This permits load-balancing strategies where faster children's results are processed immediately without waiting for slower children. Associativity ensures that the final result is independent of tree structure. This permits dynamic tree reconfiguration in response to node failures or load imbalances without affecting correctness. Idempotency enables fault tolerance under at-least-once delivery semantics. A child's result can be retransmitted and merged multiple times during network failures without corrupting the aggregate state.

The hierarchical aggregation architecture for cardinality estimation (§3.4.1) instantiates this pattern class. Worker nodes construct HyperLogLog sketches over local data partitions. Intermediate aggregators merge sketches using the register-wise maximum operation. Theorem 3.6 establishes that this operation is commutative, associative, and idempotent. Communication cost is $\Theta(ks)$ for k worker nodes transmitting sketches of size s, independent of dataset cardinality n. This fixed-cost property transforms cardinality estimation from a communication-bound problem to a computation-bound problem.

A hierarchical-reduction pattern produces correct global aggregates if and only if the oracle satisfies 3.2. The HyperLogLog analysis in §3.2 shows that the union of partial sketches is equivalent to a sketch constructed over the complete dataset. Without commutativity and associativity, the result depends on merge order. Without idempotency, duplicate transmissions corrupt the aggregate.

4.3 Structure Selection Criteria

Structure selection depends on two workload characteristics: data lifecycle dynamics and query result formation.

4.3.1 Data Lifecycle Dynamics

Static datasets perform insertion without deletion, as in append-only event logs, historical archives, and immutable data lakes. These workloads do not require mutability. Append-with-expiry datasets insert elements with time-to-live values that expire automatically. Time-windowed stream analytics, session stores with timeout, and caches with time-based eviction follow this pattern. These workloads maintain separate oracles per time window and discard entire oracles upon expiration rather than removing individual elements. This approach does not require element-level mutability.

Fully-dynamic datasets perform arbitrary insertion and deletion driven by application logic. Key-value stores with tombstones, caches with LRU eviction, and distributed hash tables with explicit remove operations exemplify this pattern. These workloads require mutability to maintain oracle consistency with dataset state.

4.3.2 Query Result Formation

Single-partition queries execute entirely within one partition, as in point lookups where routing directs queries to the responsible partition, range scans within a shard, and membership tests in node-local caches. These workloads answer queries using only local oracle state and do not require composability.

Multi-partition aggregation combines information across partitions. Global distinct counts from per-partition sketches, federated analytics across administrative domains, and datacenter-level cardinality es-

timates from server-level aggregates exemplify this pattern. These workloads require composability to merge partial results.

5 Conclusion

Disaggregated systems require compact, queryable summaries of remote state to resolve the impedance mismatch between stateless compute and network-separated storage. We demonstrated that the system-level value of probabilistic data structures derives from two properties that resolve this mismatch. The first, dynamic mutability, enables state synchronization with evolving datasets through efficient element deletion. The second, composability, enables distributed aggregation without central coordination through operations that are commutative, associative, and idempotent.

The case studies on Cuckoo Filters (§2) and HyperLogLog (§3) validated this framework. We showed that specific deployment patterns emerge as necessary consequences of these properties: co-partitioning of filter state with data for oracles optimized for mutability, and hierarchical aggregation trees for oracles optimized for composability.

The framework (§4) establishes that mutability requires element-level information retention while composability requires lossy compression through commutative operations. Theorem 4.2 formalizes this information-theoretic tension: probabilistic oracles with idempotent merge operations cannot support deletion without false negatives unless they maintain reversible element representations. This structural impossibility explains why examined structures optimize for one property. The Cuckoo Filter maintains reversible representations (explicit fingerprints) and supports deletion, but its merge operation is not idempotent. The HyperLogLog sketch has an idempotent merge operation (register-wise maximum) but discards element identities, precluding deletion. Structure selection follows from analyzing two workload characteristics: whether the dataset undergoes explicit deletion operations, and whether query results aggregate information from multiple partitions. The intersection of these characteristics determines which property the oracle must satisfy and thus which deployment pattern the system must instantiate.

Workloads requiring explicit element deletion and single-partition queries necessitate structures satisfying dynamic mutability, deployed in co-partitioned architectures where oracle state aligns with data partitioning. Workloads requiring cardinality aggregation across partitions over static or append-with-expiry datasets necessitate structures satisfying composability, deployed in hierarchical reduction architectures. Workloads requiring both properties must employ hybrid systems that maintain separate specialized oracles, as no single structure examined provides both properties optimally.

The framework exposes research directions at the intersection of the core properties with system-level constraints. Byzantine-resilient aggregation must filter malicious updates while preserving the composability required for distributed reduction. Differential privacy mechanisms must introduce noise in a manner that commutes with merge operations. Adaptive checkpointing protocols that bound error accumulation for composable oracles through threshold-based backup do not extend to structures lacking idempotent merge operations, creating an asymmetry in fault tolerance mechanisms. The incompatibility result (Theorem 4.2) establishes the fundamental tension between idempotent composability and deletion support. Open questions remain regarding relaxations of these constraints: whether bounded false negative rates during deletion enable approximate composability, whether weakened idempotence guarantees permit limited deletion support, and whether space lower bounds exist for structures that approximate both properties within specified error tolerances.

References

[1] Hazar Harmouch and Felix Naumann. Cardinality estimation: An experimental survey. *Proc. VLDB Endow.*, 11:499–512, 2017.

[2] M. Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauly, M. Franklin, S. Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Symposium on Networked Systems Design and Implementation*, pages 15–28, 2012.

[3] B. Bloom. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM*, 13:422–426, 1970.

[4] P. Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm. *Discrete Mathematics & Theoretical Computer Science*, pages 137–156, 2007.

[5] Stefan Heule, Marc Nunkesser, and Alexander Hall. Hyperloglog in practice: algorithmic engineering of a state of the art cardinality estimation algorithm. In *International Conference on Extending Database Technology*, pages 683–692, 2013.

[6] Yi Liu, Xiongzi Ge, D. Du, and Xiaoxia Huang. Parbf: A parallel partitioned bloom filter for dynamic data sets. *The International Journal of High Performance Computing Applications*, 30:259 – 275, 2014.

[7] Fay W. Chang, J. Dean, S. Ghemawat, Wilson C. Hsieh, D. Wallach, M. Burrows, Tushar Chandra, A. Fikes, and R. Gruber. Bigtable: a distributed storage system for structured data. In *USENIX Symposium on Operating Systems Design and Implementation*, pages 205–218, 2006.

[8] Sasu Tarkoma, Christian Esteve Rothenberg, and Eemil Lagerspetz. Theory and practice of bloom filters for distributed systems. *IEEE Communications Surveys & Tutorials*, 14:131–155, 2012.

[9] A. Broder and M. Mitzenmacher. Network applications of bloom filters: A survey. *Internet Mathematics*, 1:485 – 509, 2004.

[10] Sharafat Ibn Mollah Mosharraf and M. A. Adnan. Improving lookup and query execution performance in distributed big data systems using cuckoo filter. *Journal of Big Data*, 9, 2021.

[11] Paulo Sérgio Almeida, Carlos Baquero, Nuno M. Preguiça, and D. Hutchison. Scalable bloom filters. *Inf. Process. Lett.*, 101:255–261, 2007.

[12] Li Fan, P. Cao, J. Almeida, and A. Broder. Summary cache: a scalable wide-area web cache sharing protocol. In *Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, volume 28, pages 254–265, 1998.

[13] Deke Guo and Mo Li. Set reconciliation via counting bloom filters. *IEEE Transactions on Knowledge and Data Engineering*, 25:2367–2380, 2013.

[14] Bin Fan, D. Andersen, M. Kaminsky, and M. Mitzenmacher. Cuckoo filter: Practically better than bloom. *Proceedings of the 10th ACM International on Conference on emerging Networking Experiments and Technologies*, 2014.

[15] Hanhua Chen, Liangyi Liao, Hai Jin, and Jie Wu. The dynamic cuckoo filter. *2017 IEEE 25th International Conference on Network Protocols (ICNP)*, pages 1–10, 2017.

[16] A. Lakshman and Prashant Malik. Cassandra: a decentralized structured storage system. *ACM SIGOPS Oper. Syst. Rev.*, 44:35–40, 2010.

[17] Redis Labs. Redis Cluster Specification. `https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/`, 2025. Accessed: 2025-10-26.

[18] Lailong Luo, Deke Guo, Ori Rottenstreich, Richard T. B. Ma, Xueshan Luo, and Bangbang Ren. A capacity-elastic cuckoo filter design for dynamic set representation. *IEEE Transactions on Network and Service Management*, 18:4860–4874, 2021.

[19] Baozhou Luo, Wenjun Zhu, Peng Li, and Zhijie Han. Distributed dynamic cuckoo filter system based on redis cluster. *2018 IEEE 4th International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing, (HPSC) and IEEE International Conference on Intelligent Data and Security (IDS)*, pages 244–247, 2018.

[20] Bin Fan, D. Andersen, and M. Kaminsky. Memc3: Compact and concurrent memcache with dumber caching and smarter hashing. In *Symposium on Networked Systems Design and Implementation*, pages 371–384, 2013.

[21] Xiaozhou Li, D. Andersen, M. Kaminsky, and M. Freedman. Algorithmic improvements for fast concurrent cuckoo hashing. In *European Conference on Computer Systems*, pages 27:1–27:14, 2014.

[22] Wenhai Li, Zhiling Cheng, Yuan Chen, Ao Li, and Lingfeng Deng. Lock-free bucketized cuckoo hashing. In *European Conference on Parallel Processing*, pages 275–288, 2023.

[23] Stewart Grant and A. Snoeren. Cuckoo for clients: Disaggregated cuckoo hashing. In *USENIX Annual Technical Conference*, pages 959–972, 2025.

[24] Niv Dayan and M. Twitto. Chucky: A succinct cuckoo filter for lsm-tree. In *SIGMOD Conference*, 2021.

[25] Arman Mahmoudi and M. Ahmadi. Dicupit: Distributed cuckoo filter-based pending interest table. *ArXiv*, abs/2308.02801, 2023.

[26] Xiaoqiang Ding, Liushun Zhao, Lailong Luo, Junjie Xie, Deke Guo, and Jinxi Li. Gauze: enabling communication-friendly block synchronization with cuckoo filter. *Frontiers of Computer Science*, 17(3):173403, 2023.

[27] Lum Ramabaja and Arber Avdullahu. The distributed bloom filter. *ArXiv*, abs/1910.07782, 2019.

[28] Qun Huang and P. Lee. Toward high-performance distributed stream processing via approximate fault tolerance. *Proc. VLDB Endow.*, 10:73–84, 2016.

[29] I. Wilms and J. Bien. Tree-based node aggregation in sparse graphical models. *Journal of machine learning research : JMLR*, 23, 2021.

[30] Salvatore Sanfilippo. HyperLogLog in Redis. `https://antirez.com/news/75`, 2025. Accessed: 2025-10-26.

[31] Ziye Tao, G. Weber, and Y. Yu. Expected 10-anonymity of hyperloglog sketches for federated queries of clinical data repositories. *Bioinformatics*, 37:i151 – i160, 2021.

[32] P. Reviriego, Pablo Adell, and Daniel Ting. Hyperloglog (hll) security: Inflating cardinality estimates. *ArXiv*, abs/2011.10355, 2020.

[33] Tuan Le and Shana Moothedath. Byzantine resilient federated multi-task representation learning. *ArXiv*, abs/2503.19209, 2025.

[34] K. Kuwarananchareon and S. Sundaram. On the geometric convergence of byzantine-resilient distributed optimization algorithms. *SIAM J. Optim.*, 35:210–239, 2023.

[35] Otmar Ertl. Setsketch: Filling the gap between minhash and hyperloglog. *ArXiv*, abs/2101.00314, 2021.

[36] Otmar Ertl. Ultraloglog: A practical and more space-efficient alternative to hyperloglog for approximate distinct counting. *Proc. VLDB Endow.*, 17:1655–1668, 2023.

[37] Matti Karppa and R. Pagh. Hyperlogloglog: Cardinality estimation with one log more. *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[38] Jonatan Langlet, Peiqing Chen, Michael Mitzenmacher, Ran Ben Basat, Zaoxing Liu, and Gianni Antichi. Sketch disaggregation across time and space. *ArXiv*, abs/2503.13515, 2025.

[39] Zhinan Cheng, Qun Huang, and P. Lee. On the performance and convergence of distributed stream processing via approximate fault tolerance. *The VLDB Journal*, 28:821 – 846, 2018.

[40] Yifeng Zhu and Hong Jiang. False rate analysis of bloom filter replicas in distributed systems. *2006 International Conference on Parallel Processing (ICPP'06)*, pages 255–262, 2006.

[41] Giuseppe DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels. Dynamo: amazon's highly available key-value store. In *Symposium on Operating Systems Principles*, pages 205–220, 2007.

[42] Redis Labs. Cuckoo Filters in Redis. `https://redis.io/docs/latest/develop/data-types/probabilistic/cuckoo-filter/`, 2025. Accessed: 2025-10-26.